

Roteiro de apresentação — vídeo (Jenkins + Docker)

Daniel Trindade — Gerência de Configuração · UFAM

Roteiro em estilo **teleprompter** para gravar **um único vídeo** de apresentação do Trabalho 2 (Jenkins + Docker). Para cada bloco há o que **falar** (em

citação) e o que **mostrar/fazer** na tela (em **[MOSTRAR]** / **[AÇÃO]**).

- Duração alvo: **10 a 12 minutos**.
- Use a apresentação `slides/jenkins-docker.pptx` (ou no Google Slides) nos blocos de explicação, e a tela do **Jenkins + Docker Desktop** nos blocos de demonstração.
- Não precisa decorar: leia com naturalidade, pode pausar e recortar na edição.

Dica: grave os blocos de explicação (slides) e os de demonstração (Jenkins) em partes separadas e junte na edição — fica mais fácil do que tentar tudo de uma vez.

Antes de gravar (checklist rápido)

- ☐ OBS aberto; cena 1 = slides em tela cheia; cena 2 = navegador no Jenkins **com o Docker Desktop (aba Containers) num canto**.
 - ☐ Docker Desktop rodando em **Linux containers**; `docker pull node:22-alpine` já feito (para não baixar durante a gravação).
 - ☐ As 4 branches existem no GitHub e o job do Jenkins está criado.
 - ☐ Microfone testado; notificações do sistema silenciadas.
-

Bloco 1 — Abertura (~30 s)

[MOSTRAR] Slide de capa ("Jenkins + Docker").

"Olá! Nesse vídeo eu vou mostrar como adaptei um pipeline do Jenkins para rodar o **build em um container Docker** e os **testes em outro container isolado**. Vou explicar rapidamente o projeto e a arquitetura, como montei o job no Jenkins ligado ao GitHub, e no final eu demonstro os quatro cenários: tudo certo, falha de build, build instável e a build agendada. Bora lá."

Bloco 2 — O projeto (~1 min)

[MOSTRAR] Slides "API de Conversão de Temperatura" e "Arquitetura do código".

"O projeto é de propósito bem simples, para o foco ficar no pipeline e não no código. É uma API em Node.js com **dois métodos de conversão de temperatura**:

Fahrenheit para Celsius e Celsius para Fahrenheit. Ela expõe esses dois endpoints, mais uma documentação interativa."

"A estrutura é enxuta: um arquivo com a **regra de negócio** das conversões, um que **roteia as requisições**, e o servidor HTTP. Um detalhe importante: o projeto **não tem dependências externas** — usa só os módulos nativos do Node."

Bloco 3 — Casos de teste e cobertura (~1 min)

[MOSTRAR] Slide "Os dois métodos e como são validados".

"Para os testes, eu tenho **nove casos**. Os dois principais são os do enunciado: 32 graus Fahrenheit têm que dar 0 Celsius, e 0 Celsius tem que dar 32 Fahrenheit. Os outros sete cobrem os endpoints, validação de parâmetro inválido, método não permitido e rota inexistente."

"Além de passar nos testes, eu também **meço a cobertura de código**. O comando de teste gera dois relatórios: um JUnit, com o resultado de cada caso, e um lcov, com a cobertura linha a linha. Já já a gente vê isso aparecer no Jenkins."

Bloco 4 — A mudança e a arquitetura (~2 min)

[MOSTRAR] Slides "Do Trabalho 1 para o Trabalho 2", "Arquitetura do pipeline" e "O Jenkinsfile".

"No Trabalho 1, o Jenkins rodava o npm direto na máquina dele — build e teste no mesmo ambiente, sem isolamento. Agora, no Trabalho 2, cada etapa sobe o **seu próprio container Docker**: um container só para o build e outro, separado, só para o teste."

[MOSTRAR] Diagrama do pipeline.

"O fluxo é esse: o Jenkins pega o código do GitHub, sobe um primeiro container que faz a 'compilação' — que aqui é validar a sintaxe dos fontes com o `npm run check`. Depois sobe um **segundo container, limpo e isolado**, que roda os testes. No log dá pra ver os dois com **hostnames diferentes**, provando que são containers separados."

[MOSTRAR] Slide do Jenkinsfile.

"Um ponto que eu quero destacar, porque foi um aprendizado real: o Jenkins aqui roda no **Windows**. O jeito 'mágico' do plugin, o `agent docker`, **não funciona** nesse caso — ele tenta montar o caminho do Windows dentro de um container Linux e quebra. Por isso eu uso `docker run` explícito, controlando o volume e o diretório de trabalho. Vantagem: **nem precisa do plugin Docker Pipeline**, só do Docker instalado."

"E tem mais um detalhe de Windows: o serviço do Jenkins **não consegue escrever** no volume montado. Então os relatórios são gravados **dentro do container** e eu os trago de volta com `docker cp`. É o que faz o JUnit e a cobertura aparecerem mesmo nesse ambiente."

Bloco 5 — Montando o job no Jenkins (~1 min)

[MOSTRAR] Slides "Pré-requisitos", "Montar o job" e "Ligação com o GitHub". Em seguida, **a tela real do job em Configure**.

"Para montar isso no Jenkins: eu crio um item do tipo **Pipeline**, e na definição escolho **'Pipeline script from SCM'**. Aí aponto o **Git** para a URL do meu repositório no GitHub, escolho a branch, e digo que o script é o **Jenkinsfile** na raiz do projeto."

[AÇÃO] Mostre na tela: Repository URL, Branch Specifier, Script Path = Jenkinsfile.

"Pronto — é isso que liga o Jenkins ao GitHub. A partir daqui, é só disparar. Vamos aos cenários."

Bloco 6 — Demonstração dos cenários (~5 min)

Transição de fala: "Agora a parte que interessa: vou rodar quatro cenários e mostrar como o Jenkins se comporta em cada um."

Cenário 1 — "Tudo certo!" (SUCCESS)

[AÇÃO] Em Configure, Branch Specifier = `*/conversion-cenario-build-sucesso`, Salvar. Clicar **Build Now**.

"No primeiro cenário está tudo certo. Eu aponto para a branch de sucesso e mando **Build Now**."

[MOSTRAR] Docker Desktop: o container de build sobe e some; depois o de teste.

"Olha aqui no Docker: subiu o primeiro container, o de build. Quando ele termina, sobe um segundo, o de teste."

[MOSTRAR] Console Output: os blocos `=== BUILD ===` e `=== TESTE ===` com **hostnames diferentes**; a linha do `docker cp`.

"E aqui está a prova do isolamento: o build rodou neste container, o teste neste outro — IDs diferentes. Repare também no `docker cp` trazendo os relatórios pra fora do container."

[MOSTRAR] Resultado **SUCCESS** + Test Result (9 testes).

"Resultado: **SUCCESS**. Os nove testes passaram. Build verde."

Cenário 2 — "Deu ruim!" (FAILURE)

[AÇÃO] Branch Specifier = */conversion-cenario-falha-sintaxe, Salvar, Build Now.

"No segundo cenário, eu deixei de propósito um **erro de sintaxe** no código — uma desestruturação sem fechar a chave."

[MOSTRAR] Console Output: o build falhando no `npm run check` com `SyntaxError`; e a linha `Stage "Test" skipped`.

"O container de build sobe, roda o check e **quebra na compilação** — esse é o `SyntaxError`. E olha que importante: o estágio de teste **nem chega a rodar**. Como o build falhou, o Jenkins pula o teste; o segundo container nem sobe."

[MOSTRAR] Resultado **FAILURE** (bola vermelha).

"Resultado: **FAILURE**. A falha foi na etapa de build, exatamente como esperado."

Cenário 3 — "Tá instável!" (UNSTABLE)

[AÇÃO] Branch Specifier = */conversion-cenario-testes-falhando, Salvar, Build Now.

"No terceiro cenário, o código **compila** normalmente, mas eu errei a fórmula da conversão de propósito — troquei o cinco nonos por quatro nonos. Então o build passa, mas um teste falha."

[MOSTRAR] Console: build verde; depois o teste com um caso falhando.

"Viu? O container de build passou. Já no container de teste, um caso falha — a conversão não bate com o valor esperado."

[MOSTRAR] Resultado **UNSTABLE** (amarelo) + Test Result com o teste vermelho.

"E o resultado aqui é **UNSTABLE**, não **FAILURE**. Eu uso um `catchError` que captura a falha do teste e marca o build como **instável**, mas ainda publica o relatório JUnit — por isso dá pra ver qual teste quebrou."

Cenário 4 — "O lendário nightly" (SUCCESS, agendado)

[AÇÃO] Branch Specifier = */conversion-cenario-build-agendado, **Save**. **Não** clicar em Build Now.

"O último cenário é o **nightly**, a build agendada. Nessa branch eu configurei um cron que dispara o job sozinho, a cada minuto, só para a demonstração."

[MOSTRAR] A tela do job parada, esperando. Quando iniciar sozinho, abrir o build.

"Repare que eu **não vou clicar em nada**. Vou só esperar o horário chegar... e pronto — o build começou **sozinho**."

[MOSTRAR] Topo do Console Output: **"Started by timer"**.

"E aqui está a prova: '**Started by timer**'. Foi o agendador que disparou, não eu. O fluxo é o mesmo de sempre, container de build e container de teste, e o resultado é **SUCCESS**, automático."

Lembrete (não precisa falar): depois de gravar, volte o Branch Specifier para */main ou desabilite o job, senão ele continua rodando a cada minuto.

Bloco 7 — Cobertura de código (~30 s)

[MOSTRAR] Console Output de um build de sucesso, na **tabela de cobertura**; e os **artefatos arquivados** (reports/lcov.info, reports/junit.xml).

"Fechando a parte técnica: além de passar nos testes, cada execução também **mede a cobertura de código**, linha a linha. O Jenkins arquiva os dois relatórios — o JUnit, com o resultado dos testes, e o lcov, com a cobertura."

Bloco 8 — Encerramento (~30 s)

[MOSTRAR] Slide de conclusão.

"E é isso! A gente saiu de um pipeline que rodava tudo no agente para um que roda build e testes em **containers Docker isolados** — com mais reprodutibilidade e isolamento, e funcionando até num Jenkins no Windows. O código e o passo a passo estão no repositório, no link da descrição. Valeu, e até a próxima!"

Dicas de gravação e publicação

- **OBS:** grave em 1080p, 30 fps. Deixe o zoom do navegador em ~110–125% para o texto do Console Output ficar legível.
- **Tamanho de fonte do Jenkins:** aumente o zoom antes de gravar os logs — os hostnames e o Started by timer precisam estar nítidos.
- **Edição:** corte os tempos de espera (pull de imagem, fila). No Cenário 4, pode acelerar a espera do cron.
- **YouTube:** suba como "**Não listado**" se for só para entrega; cole o link no slide de conclusão e no de cada cenário.
- **Capítulos no YouTube (opcional):** coloque marcações na descrição — 0:00 Intro, 0:30 O projeto, 2:30 Arquitetura, 4:00 Montando o job, 5:00 Cenários, etc. — facilita para o avaliador.